Spec 001 — Open Overbid Window after Bid ≥10

Status: Draft — awaiting review **Authored:** 2026-04-19 **Touches:** bidding phase, partner reveal timing, AI bid logic, `BiddingPanel.vue`

1. Problem

Today, whoever bids ≥10 first closes bidding round 1 instantly. The partner's hand is revealed the same moment, and no other seat gets any voice. This is a race — a speculative 10 from the first seat denies the other three seats any chance to outbid with a stronger hand. It also makes asking-for-support asymmetric (only pre-bid seats ever ask).

2. Proposed Change

After a player bids ≥10, open a **single clockwise overbid window** covering the remaining 3 seats. Each other seat, in clockwise order from the bidder, may either:

- **Pass** — yield to the current bid, OR
- **Raise** — bid at least `currentBid + 1`, pick any trump suit (may differ from current), becoming the new bidding leader.

A raise **does not reset** the window — it continues clockwise. Each of the 3 non-bidder seats gets **at most one action** per window. Once all three have acted (passed or been passed through after raising), the window closes and the game transitions to `partner_reveal` for the **final** bidder.

Partner-reveal UI fires only at the close. That is the core user-visible change: **the partner's hand is not shown until the overbid window has fully resolved.**

2a. Scope — what stays the same

- Bidding round 1 itself (pre-first-bid passes): unchanged.
- Forced bid rule (all 4 pass + seat-left-of-dealer must bid ≥8): unchanged.
- Forced bid of 8 or 9 → straight to `playing`, no partner reveal, **no overbid window** (window only opens when a bid ≥10 is made).
- Forced bid of 10+: overbid window opens, same as voluntary ≥10.
- Bidder's own post-reveal `ask_support` action: unchanged (still allowed in partner-reveal).
- Support signals, TBAA, scoring, dealer rotation, win conditions: unchanged.
- Bid 13 semantics (`docs/GameTime.md` §Bid-13 rule): unchanged — if the final bid after overbid closes is 13, normal bid-13 handling applies.

2b. Scope — what changes

- New phase `bidding_raise` between `bidding` / `bidding_forced` and `partner_reveal`.
- New socket events `raise_bid`, `pass_raise` (following existing `increase_bid` / `pass_bid` handlers) that event is coupled to the bidder-only partner-reveal semantics in both server and client; overbidding risks regressions.)
- Bid history records raises and raise-passes distinctly so the UI can show them.
- `partnerHand` in `getStateFor()` stays hidden during `bidding_raise` and only appears once the phase moves to `partner_reveal`.
- AI gets a new decision branch for the raise window.

3. State-Machine Diff

Before

```
bidding ──bid≥10──► partner_reveal ──start_play──► playing
bidding ──4 pass──► bidding_forced ──bid≥10──► partner_reveal
                                     └bid 8/9──► playing
```

After

```
bidding ──bid≥10──► bidding_raise ──3 seats act──► partner_reveal ──start_play──► playing
                    │  └ on each seat's turn: pass_raise | raise_bid
                    │    raise_bid: bid, trump, biddingSeat, biddingTeam all update;
                    │                window continues clockwise, does NOT reset
bidding ──4 pass──► bidding_forced ──bid≥10──► bidding_raise ──3 seats act──► partner_reveal
                                     └bid 8/9──► playing   (unchanged)
```

Invariant: during `bidding_raise`, `partnerHand` is null in every seat's state view. Only after transition to `partner_reveal` does it populate for the final `biddingSeat`'s partner.

4. Server Changes — `server/gameLogic.js`

4a. New state fields on `BurrisGame`

- `raiseSeatsRemaining: number[]` — ordered list of seats still to act in the current raise window. Populated on entry to `bidding_raise`, consumed on each pass/raise action.

4b. Modify `placeBid()` — `gameLogic.js:498`

At the branch that currently sets phase = `'partner_reveal'` when `bid ≥ 10` (lines 523–525), instead:

```
if (bid ≥ 10)
  // Open overbid window — every other seat, clockwise from bidder
  this.phase = 'bidding_raise';
  this.raiseSeatsRemaining = [1, 2, 3].map(n => (seat + n) % 4);
  this.activeSeat = this.raiseSeatsRemaining[0];
} else {
  // bid 8 or 9 (forced only) -> unchanged
  this.phase = 'playing';
  this.activeSeat = this.dealer + 1) % 4;
}
```

4c. New method `raiseBid(seat, bid, trump)`

Validations:

- phase `bidding_raise`
- `activeSeat` `== seat`
- `bid > this.bid` and `bid ≤ 13`
- `SUITS.includes(trump)` — raiser may pick any trump, including the same as current bidder.

On success: update `this.bid`, `this.trump`, `this.biddingSeat = seat`, `this.biddingTeam = seat % 2`, push `{ seat, action: 'raise', bid, trump }` to `biddingHistory`. Then shift the front of `raiseSeatsRemaining` (the seat that just acted) and advance the window — see 4e.

4d. New method `passRaise(seat)`

Validations: phase `bidding_raise`, `activeSeat` `== seat`. Push `{ seat, action: 'pass_raise' }` to `biddingHistory`. Shift `raiseSeatsRemaining` and advance — see 4e.

4e. Window advancement helper (used by both 4c and 4d)

```
// After popping the acted seat off the head:
this.raiseSeatsRemaining.shift();
if (this.raiseSeatsRemaining.length == 0) {
  this.phase = 'partner_reveal';
  this.activeSeat = this.biddingSeat;  // final bidder controls reveal
} else {
  this.activeSeat = this.raiseSeatsRemaining[0];
}
```

Note: `raiseSeatsRemaining` is a fixed-size queue seeded when the window opens. A raise **does not** re-seed it — each seat gets exactly one chance regardless of how many raises happen.

4f. `getStateFor(seat)` — `gameLogic.js:1058`

Gate the existing `partnerHand` reveal on phase `== 'partner_reveal'` || phase `== 'playing'` (not `bidding_raise`). Currently the reveal is tied to `bid ≥ 10` — add the phase guard so raising window stays blind.

4g. `startPlay()` — `gameLogic.js:599`

No change. Still guards on phase `== 'partner_reveal'`. The `biddingSeat` field will correctly reflect the final (possibly raised) bidder.

5. Server Changes — `server/server.js`

Add two socket handlers, modelled on the existing `increase_bid` / `pass_bid` handlers:

- `raise_bid { bid, trump }` → `game.raiseBid(socket.seat, bid, trump)`
- `pass_raise {}` → `game.passRaise(seat)`

Both re-broadcast state and call `checkAiTurn()` after success. Both reject if the calling socket's seat is not `game.activeSeat` (same pattern as existing handlers).

6. Client Changes — `client/src/components/BiddingPanel.vue`

6a. New phase branch in the template

Add a block for phase `== 'bidding_raise'` that renders:

- When `isMyTurn`: the existing ≥10 bid-picker (suit + number) with **min bid** = `currentBid + 1`, plus a "Pass" button and a "Raise" button. Reuses 90% of the existing bidding turn template — extract to a sub-template or use a computed-driven flag that already accounts for raise mode.
- When not my turn: "Waiting for {seatName(activeSeat)} to raise or pass…" with the bid-history bubbles above.

6b. `aiBid` computed update

```
const minBid = computed(() => {
  if (phase.value == 'bidding_raise') return (currentBid.value ?? 10) + 1;
  return (phase.value == 8 ? 10:
});
```

6c. Emit two new events up to `GameBoard.vue` → socket

`raise-bid { bid, trump }` and `pass-raise {}` on `GameBoard.vue` wires them to `socket.emit('raise_bid', …)` / `socket.emit('pass_raise')`.

6d. `historyLabel()` — extend

Add cases for `action == 'raise'` (render like a bid, with an up-arrow prefix) and `action == 'pass_raise'` (render "Pass raise" in slate-400).

6e. Partner hand reveal — already safe

`PlayerHand` renders `partnerHand` based on server state. Because `getStateFor()` now gates it on phase, no client-side change is needed — partner cards simply won't appear until `partner_reveal`.

7. AI Changes — `server/aiPlayer.js`

Extend `decideAction()` (around line 206) with a new case:

```
if (phase == 'bidding_raise' && activeSeat == seat) {
  return this._decideRaise();
}
```

7a. `_decideRaise()` logic

Re-use the existing hand-evaluation that `_decideBid()` uses. If the bot's own estimated tricks ≥ `currentBid + 1`, have it raise by 1 to that amount with its preferred trump. Otherwise, pass. Do **not** consider partner's hand — it is not revealed yet.

For bots: the simplest safe heuristic is "only raise if you would have opened at `currentBid+1` yourself". More sophisticated strategies can come later.

7b. Existing `_decideIncrease()` — unchanged

Still only fires in `partner_reveal`, for the final bidder, with full combined-hand view. Behavior preserved.

8. Docs Changes

- `docs/GameTime.md` §3c: Rename to "3c. Overbid Window (bid ≥10)" and describe the new window. Move partner-reveal steps into a new §3d.
- `docs/SocketAPI.md`: document `raise_bid` and `pass_raise` events (both directions — client emit, server broadcast deltas).
- `docs/Board.md`: note that the bidding panel now has a third mode ("raise or pass"), and partner cards appear only after the window closes.
- `docs/AILogic.md`: add `_decideRaise()` section.

9. Test Cases

To be added to `server/test-game.js` scaffolding, plus new unit-level scenarios. The existing test harness drives 1 human + 3 bots through full games; the scenarios below require seeded hands or direct calls into `BurrisGame`, not just the end-to-end runner. A new `server/test-bidding.js` (or similar) that exercises `BurrisGame` directly is the path of least resistance.

9a. Happy-path overbid

1. Seat 1 bids 10 ♠ (voluntary, round 1).
2. State: phase `== 'bidding_raise'`, `activeSeat == 2`, `raiseSeatsRemaining == [2, 3, 0]`, `partnerHand == null` for all seats.
3. Seat 2 pass_raise. `activeSeat == 3`. Still `bidding_raise`.
4. Seat 3 `raise_bid` ♥, `trump == 'H'`, `bid == 11`, `trump == 'H'`, `biddingSeat == 3`, `biddingTeam == 1`, `activeSeat == 0`. Still `bidding_raise`.
5. Seat 0 pass_raise. `activeSeat == 3` (hidden). Now phase `== 'partner_reveal'`.
6. `getStateFor(seat)` now shows seat 3's hand (seat 3's partner) as `partnerHand`. Seat 1's original 10-bid hand is exposed.
7. Seat 3 `start_play`. Phase `== playing`, seat 3 leads.

Expected: final `biddingSeat == 3`, `bid == 11`, `trump == 'H'`, only seat 3's partner's (seat 1) hand is in `partnerHand`, never seat 3's before window closed.

9b. All pass the raise

1. Seat 1 bids 10 ♠.
2. Seats 2, 3, 0 all pass_raise.
3. Seat `biddingSeat == 1`, `bid == 10`, `trump == 'S'`, phase `== 'partner_reveal'`. Partner (seat 3) hand revealed.

9c. Multiple raises in one window

1. Seat 1 bids 10 ♠.
2. Seat 2 raises to 11 ♦.
3. Seat 3 raises to 12 ♣.
4. Seat 0 passes.
5. Final: `biddingSeat == 3`, `bid == 12`, `trump == 'C'`, `raiseSeatsRemaining` was consumed exactly 3 times. Seat 2 never gets a second chance.

9d. Forced-bid 8 or 9 — no window opens

1. All 4 pass in round 1.
2. `biddingForced` — seat-left-of-dealer forced to bid 8 ♠.
3. Transition straight to `playing`. No `bidding_raise`, no partner reveal. (Regression test — ensure the new phase did not leak into the 8/9 branch.)

9e. Forced-bid ≥10 — window opens

1. All 4 pass in round 1.
2. Forced seat bids 10 ♥.
3. `bidding_raise` opens covering the other 3 seats. Same flow as 9a from there.

9f. Raise to 13

1. Seat 1 bids 10 ♠.
2. Seat 2 raises to 13 ♥.
3. Seats 3 and 0 pass.
4. Final: `biddingSeat == 2`, `bid == 13`. Existing bid-13 dealer-rotation rules apply at round end — **nothing special about the path through the window**.

9g. Cannot re-bid after pass_raise

Seat 2 passes the raise. Later in the same window, seat 2 attempts `raise_bid`. Server rejects with "Not your turn" because `raiseSeatsRemaining` no longer contains seat 2.

9h. Cannot raise below currentBid+1

Seat 1 bids 10. Seat 2 attempts `raise_bid` with bid = 10. Server rejects with "Bid must be greater than current bid" (or similar). Existing `increase_bid` guard copied.

9i. Partner hand stays hidden mid-window

After seat 1 bids 10 ♠ and seat 2 raises to 11 ♦, before seat 3 acts: `getStateFor(3).partnerHand` `== null`, `getStateFor(1).partnerHand` `== null`, `getStateFor(2).partnerHand` `== null`. (Bidder-changes-mid-window invariant.)

9j. Disconnect during raise window

Seat 2 (human) disconnects while holding the raise turn. Bot replaces them and calls `_decideRaise()`. Game continues without stall. (Integration with existing disconnect-to-bot logic in `server.js`.)

9k. Bidder's own post-reveal increase still works

After window closes with final bidder seat 3 at 11 ♥, seat 3 uses the existing `increase_bid` flow in `partner_reveal` to go to 12 ♥. Unchanged behaviour.

9l. AI-only game runs to completion

With `FAST_TEST=1 node server/test-game.js` and 4 bots, the new phase cannot deadlock — `_decideRaise()` must always either raise or pass. Run 3 full games; assert no timeouts, no stuck phases.

10. Open Questions

- **Ask-partner-for-support during raise window**: spec says no (raising seat has not asked in round 1, and re-asking mid-window adds UI complexity). Confirm the user wants this omitted. Yes
- **Should the bid-number picker in the raise window show the current bidder's chosen trump suit as default, or force an explicit pick?** Spec defaults to "explicit pick" (same as round 1), which is safer but one extra tap.
- **Raising back to the same team**: if seat 1 (Team A) bids 10 and seat 3 (also Team A) raises to 11, is that legal? Current spec says yes — biddingTeam just stays 0. Confirm this is desired; an alternative is to restrict raises to the opposing team only. Yes
  Legal Yes, they can but possible

11. Verification

Once implemented:

1. `cd server && npm run dev` + `cd client && npm run dev`; play a round from the browser, bid 10, confirm the other seats are offered raise/pass UI before the partner hand appears.
2. `FAST_TEST=1 node server/test-game.js` — confirm 3 AI-driven games complete, partner hand never leaks during raise window (assert in the test-game harness or a new scenario file).
3. Unit: new `server/test-bidding.js` covering 9a–9k.
4. Visual regression: partner cards should remain face-down on the north side during the raise window; flip to face-up on transition to `partner_reveal`.

12. Critical Files

Path	Role
`server/gameLogic.js`	New phase + `raiseBid` / `passRaise`; `placeBid` branch change; `getStateFor` guard
`server/server.js`	Two new socket handlers
`server/aiPlayer.js`	New `_decideRaise` branch
`client/src/components/BiddingPanel.vue`	New phase UI branch, minBid update, emits
`client/src/components/GameBoard.vue`	Wire new emits to socket events
`docs/GameTime.md`, `docs/SocketAPI.md`, `docs/Board.md`, `docs/AILogic.md`	Doc updates
`server/test-bidding.js`	New — unit-tish scenarios 9a–9k